

ODC-GUINÉE

Formation Web Full-Stack — 8 avril au 25 juin 2026

Cahier de Conception Backend

MedConnect ODC — API REST de Gestion Médicale

Projet	Projet 1 — Plateforme de prise de rendez-vous médical
Langage / Framework	Java 17+ / Spring Boot 3
Sécurité	Spring Security 6 — JSON Web Tokens (JWT) Stateless
Persistance	Spring Data JPA / Hibernate
Base de données	MySQL
Gestionnaire de dépendances	Maven

Sommaire

1. Choix Technologiques et Architecture Globale
2. Architecture Logicielle en Couches
3. Spécifications de la Couche Modèle
4. Spécifications de la Couche Accès aux Données (Repositories)
5. Spécifications de la Couche Métier (Services)
6. Spécifications de la Couche Contrôleurs (Endpoints REST)
7. Sécurité Applicative et Filtrage
8. Gestion des Erreurs et Contrats de Données (DTO)
9. Build et Déploiement
10. Conclusion

1. Choix Technologiques et Architecture Globale

Le backend de MedConnect ODC a été conçu selon une architecture découplée (client-serveur), communiquant exclusivement avec le frontend React via des interfaces de programmation REST sécurisées par jeton JWT. Ce choix garantit une indépendance totale entre les couches de présentation et de traitement métier, facilitant la maintenabilité, les tests automatisés et le déploiement indépendant de chaque composant.

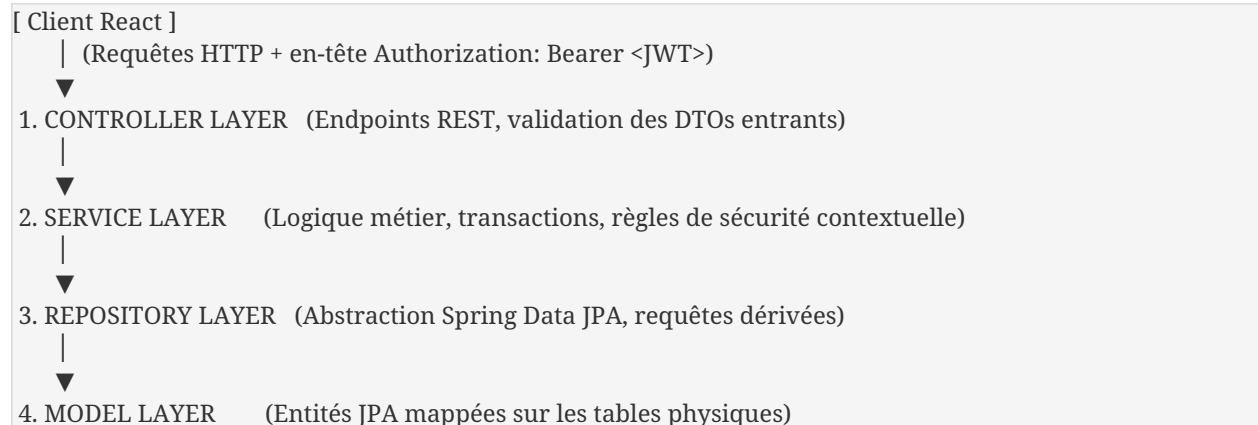
Élément	Choix retenu
Langage & Framework	Java 17+ / Spring Boot 3
Sécurité	Spring Security 6 et JSON Web Tokens (JWT) Stateless
Gestionnaire de dépendances	Maven
Persistence des données	Spring Data JPA / Hibernate
Base de données	MySQL
Validation des entrées	Jakarta Bean Validation (annotations DTO)
Hachage des mots de passe	BCryptPasswordEncoder
Build & exécution	mvn clean package → JAR exécutable autonome

1.1. Principes directeurs

- Statelessness : aucune session n'est conservée côté serveur ; chaque requête authentifiée transporte son propre jeton JWT.
- Séparation des responsabilités : chaque couche (Controller, Service, Repository, Model) n'a qu'une seule raison de changer.
- Sécurité par défaut : tout endpoint est protégé sauf déclaration explicite en zone publique (/api/auth/**).
- Transactionnalité : toute opération métier modifiant plusieurs entités est encapsulée dans une transaction @Transactional pour garantir l'atomicité.

2. Architecture Logicielle en Couches

Le code source respecte une architecture en couches strictes afin de garantir la séparation des responsabilités, la maintenabilité et l'extensibilité du projet. Chaque requête HTTP entrante traverse successivement les quatre couches applicatives suivantes :



2.1. Rôle de chaque couche

Couche	Responsabilité
Controller	Expose les endpoints REST, déséréalise et valide les DTOs entrants (@Valid), délègue immédiatement à la couche Service, gère les codes de statut HTTP.
Service	Porte l'ensemble des règles de gestion métier, orchestre les appels aux repositories, gère les transactions et les contrôles de sécurité contextuelle (ex. vérifier que le médecin connecté est bien propriétaire du rendez-vous).
Repository	Interfaces étendant JpaRepository, exposant des requêtes dérivées du nom de méthode ou des requêtes JPQL personnalisées.
Model (Entité)	Représentation objet-relationnelle des tables physiques de la base de données, annotée avec JPA/Hibernate.

3. Spécifications de la Couche Modèle

Pour répondre à l'exigence d'une base de données normalisée en 3FN tout en conservant un code Java orienté objet propre, la stratégie d'héritage par jointure (`InheritanceType.JOINED`) a été retenue pour les profils utilisateurs.

Entité	Description
User (classe mère)	Centralise les informations d'authentification : <code>idUser</code> , email, password (haché), role, <code>estActif</code> .
Patient (classe fille)	Étend User par jointure pour encapsuler les attributs spécifiques aux patients : <code>idPatient</code> , <code>numeroSecu</code> , antécédents médicaux.
Medecin (classe fille)	Étend User par jointure pour encapsuler les informations professionnelles : <code>idMedecin</code> , matricule, relation <code>ManyToOne</code> vers <code>Specialite</code> .
Specialite	Gère les domaines d'expertise médicale des médecins (référentiel).
Disponibilite	Représente un créneau horaire ouvert par un médecin, avec son statut <code>estLibre</code> .
RendezVous	Gère les réservations associant un Patient et une Disponibilite spécifique, avec un statut de cycle de vie.
Consultation	Fiche historique médicale (diagnostic, notes, ordonnance) liée de manière unique (1:1) à un rendez-vous honoré.
Notification	Trace le flux d'alertes applicatives pour l'ensemble des utilisateurs de la plateforme.

3.1. Justification de l'héritage par jointure

Cette stratégie (`@Inheritance(strategy = InheritanceType.JOINED)`) permet de conserver une table utilisateurs unique pour l'authentification (login, rôle, statut), tout en isolant dans des tables filles distinctes (patients, medecins) les attributs propres à chaque métier. Elle évite ainsi la prolifération de colonnes nullable dans une table unique (anti-pattern de la table générique) tout en conservant la possibilité de manipuler un objet User polymorphe côté code Java (ex. dans le contexte de sécurité Spring).

4. Spécifications de la Couche Accès aux Données (Repositories)

Les interfaces étendent `JpaRepository<Entity, ID>` pour s'affranchir du code SQL brut tout en tirant parti de la puissance des requêtes dérivées de Spring Data JPA, qui génère automatiquement l'implémentation à partir du nom de la méthode.

Repository	Méthodes clés
UserRepository	<code>findByEmail(String email)</code> — recherche par email pour le processus d'authentification.
MedecinRepository	<code>findByUser_Email(String email)</code> — extraction du profil métier à partir de l'email de l'utilisateur connecté.
PatientRepository	<code>findByUser_Email(String email)</code> — extraction du profil métier à partir de l'email de l'utilisateur connecté.
DisponibiliteRepository	<code>findByMedecinAndEstLibreTrue(...)</code> — filtrage des créneaux actifs et libres pour un médecin donné.
RendezVousRepository	<code>findByPatient_IdPatient(...)</code> / <code>findByDisponibilite_Medecin_IdMedecin(...)</code> — suivi des plannings par identifiant de profil métier.
ConsultationRepository	<code>findByRendezVous_IdRdv(...)</code> — récupération de la fiche médicale liée à un rendez-vous.
NotificationRepository	<code>findByUser_IdUserOrderByDateEnvoiDesc(...)</code> — historique des alertes d'un utilisateur, du plus récent au plus ancien.

5. Spécifications de la Couche Métier (Services)

Les services portent l'ensemble des règles de gestion et intègrent l'annotation @Transactional afin de garantir l'intégrité des données en cas d'erreur survenant au cours d'une opération composite.

5.1. AuthService

- registerPatient : inscription publique. Crée simultanément l'entité User (rôle PATIENT, mot de passe haché par BCrypt) et son profil Patient associé, puis génère un token JWT initial retourné au client.
- authenticateUser / authenticateAdmin : vérification des identifiants (email + mot de passe, et clé secrète pour l'administrateur) et retour d'un jeton JWT incluant le rôle de l'utilisateur dans ses claims.
- forgotPassword / resetPassword : génération d'un jeton de réinitialisation à usage unique avec expiration, envoyé par email, puis validation et mise à jour sécurisée du mot de passe.

5.2. MedecinService

- declareDisponibilite : calcule automatiquement la durée en minutes entre l'heure de début et de fin, puis persiste le créneau.
- changerStatutRdv : permet la confirmation ou l'annulation d'un rendez-vous. En cas d'annulation, le créneau de disponibilité associé est automatiquement rouvert (estLibre = true).
- redigerConsultation : permet la saisie définitive de la fiche médicale. Une barrière de sécurité vérifie que le médecin connecté est bien le propriétaire du rendez-vous avant d'autoriser l'écriture.

5.3. PatientService

- prendreRendezVous : réserve un créneau libre. La disponibilité passe instantanément à estLibre = false pour bloquer toute concurrence d'accès au même créneau.
- annulerRendezVous : met à jour le statut du rendez-vous et libère la plage horaire pour les autres patients.
- consulterHistorique : restitue l'ensemble des consultations passées du patient connecté, avec diagnostic et ordonnances.

5.4. AdminService

- creerCompteMedecin : création exclusive d'un compte médecin par l'administrateur, avec affectation obligatoire d'une spécialité.
- activerDesactiverUtilisateur : bascule le statut estActif d'un compte, sans suppression physique des données (historisation pour audit).

- `superviserPlateforme` : agrège les statistiques globales (nombre d'utilisateurs, rendez-vous en cours, taux de confirmation, etc.).

6. Spécifications de la Couche Contrôleurs (Endpoints REST)

Les contrôleurs exposent les endpoints de l'application et sécurisent l'injection du contexte de l'utilisateur connecté via l'objet Authentication fourni par Spring Security.

6.1. Endpoints publics — `/api/auth/**`

Méthode	Endpoint	Description
POST	<code>/api/auth/register</code>	Inscription d'un nouveau patient.
POST	<code>/api/auth/login</code>	Connexion générale (Patient / Médecin) — retourne un JWT.
POST	<code>/api/auth/admin/login</code>	Connexion hautement sécurisée de l'administrateur (+ clé secrète).
POST	<code>/api/auth/forgot-password</code>	Envoi d'un email de réinitialisation de mot de passe.
POST	<code>/api/auth/reset-password</code>	Réinitialisation du mot de passe via le jeton reçu par email.

6.2. Endpoints Patients — `/api/patient/**`

Méthode	Endpoint	Description
GET	<code>/api/patient/disponibilites</code>	Liste des créneaux médicaux vacants.
GET	<code>/api/patient/medecins</code>	Liste des médecins disponibles, avec filtre par spécialité.
POST	<code>/api/patient/rendez-vous</code>	Réservation d'une consultation.
GET	<code>/api/patient/rendez-vous</code>	Liste de ses rendez-vous programmés.
PATCH	<code>/api/patient/rendez-vous/{id}/annuler</code>	Annulation d'une réservation.
GET	<code>/api/patient/historique</code>	Consultation de son dossier médical et de ses ordonnances passées.

6.3. Endpoints Médecins — /api/medecin/**

Méthode	Endpoint	Description
POST	/api/medecin/disponibilites	Déclaration d'un créneau de travail.
GET	/api/medecin/disponibilites	Liste de ses propres disponibilités créées.
GET	/api/medecin/rendez-vous	Liste des patients attendus (agenda).
PATCH	/api/medecin/rendez-vous/{id}/confirmer	Validation d'un rendez-vous en attente.
PATCH	/api/medecin/rendez-vous/{id}/annuler	Annulation d'un rendez-vous.
POST	/api/medecin/rendez-vous/{id}/consultation	Clôture du rendez-vous et rédaction du rapport médical.

6.4. Endpoints Administration — /api/admin/**

Méthode	Endpoint	Description
GET	/api/admin/users	Visibilité à 360° sur l'ensemble des comptes de la plateforme.
POST	/api/admin/medecins	Création exclusive et sécurisée d'un compte médecin.
PATCH	/api/admin/users/{id}/desactiver	Désactivation d'un utilisateur (compte bloqué, données historisées).
PATCH	/api/admin/users/{id}/activer	Réactivation d'un compte suspendu.
GET	/api/admin/rendez-vous	Surveillance en temps réel de l'ensemble du planning de la structure.

6.5. Endpoints transverses

Méthode	Endpoint	Description
GET	/api/notifications	Notifications de l'utilisateur connecté.
PATCH	/api/notifications/read-all	Marquer toutes les notifications comme lues.
GET	/api/users/profile	Récupération du profil de l'utilisateur connecté.
PUT	/api/users/profile	Modification du profil (coordonnées, photo).

7. Sécurité Applicative et Filtrage

La sécurité du backend repose sur une configuration Stateless stricte, sans gestion de session côté serveur, conformément aux bonnes pratiques des API REST modernes.

7.1. Hachage des mots de passe

Aucun mot de passe n'est stocké en clair. Utilisation systématique de BCryptPasswordEncoder, un algorithme de hachage adaptatif et résistant aux attaques par force brute, intégré nativement à Spring Security.

7.2. JwtAuthenticationFilter

Ce filtre intercepte chaque requête entrante, extrait l'en-tête Authorization: Bearer <token>, valide l'authenticité de la signature et la date d'expiration du jeton, puis injecte les droits et rôles de l'utilisateur dans le SecurityContext de Spring, le rendant disponible pour toute la suite du traitement de la requête.

7.3. Protection par rôle

La configuration Spring Security (SecurityFilterChain) restreint l'accès à chaque famille d'endpoints selon le rôle porté par le JWT : /api/patient/** exige le rôle PATIENT, /api/medecin/** exige le rôle MEDECIN, et /api/admin/** exige le rôle ADMIN. Toute tentative d'accès non autorisée retourne un code HTTP 403 Forbidden.

7.4. CORS et CSRF

Le mécanisme CSRF est désactivé car l'authentification n'utilise pas les cookies du navigateur mais un token volatile transmis dans l'en-tête HTTP, immunisant naturellement l'API contre les attaques de type contrefaçon de requête intersite (CSRF). La configuration CORS autorise explicitement l'origine du frontend React en production comme en développement.

8. Gestion des Erreurs et Contrats de Données (DTO)

Afin de ne jamais exposer directement les entités JPA au client (et d'éviter les problèmes de sérialisation en boucle liés aux relations bidirectionnelles), chaque endpoint communique via des objets de transfert de données (DTO) dédiés en entrée comme en sortie.

- RequestDTO : validé par les annotations Jakarta Bean Validation (@NotNull, @Email, @Size, etc.) avant tout traitement métier.
- ResponseDTO : ne restitue que les champs strictement nécessaires au frontend, en excluant notamment le mot de passe haché.
- GlobalExceptionHandler (@ControllerAdvice) : centralise la capture des exceptions métier et techniques, et retourne des réponses JSON normalisées avec le code HTTP approprié (400, 401, 403, 404, 409, 500).

8.1. Exemple de structure de réponse d'erreur

```
{
  "timestamp": "2026-06-30T10:15:00Z",
  "status": 403,
  "error": "Forbidden",
  "message": "Vous n'êtes pas autorisé à modifier ce rendez-vous.",
  "path": "/api/medecin/rendez-vous/42/consultation"
}
```

9. Build et Déploiement

- Compilation : mvn clean package génère un JAR exécutable autonome.
- Exécution : java -jar medconnect-backend.jar
- Variables d'environnement requises : SPRING_DATASOURCE_URL, SPRING_DATASOURCE_USERNAME, SPRING_DATASOURCE_PASSWORD, JWT_SECRET (clé forte, minimum 32 caractères).
- Gestion du schéma : spring.jpa.hibernate.ddl-auto=update en développement, valide en production (le schéma physique étant alors entièrement piloté par le script SQL versionné).

10. Conclusion

L'architecture backend de MedConnect ODC, fondée sur Spring Boot 3 et une séparation stricte en quatre couches, garantit à la fois la robustesse fonctionnelle attendue d'une application médicale (intégrité transactionnelle, traçabilité, contrôle d'accès par rôle) et une base de code maintenable et testable, prête à évoluer vers de nouvelles fonctionnalités telles que la téléconsultation ou la messagerie interne.